

Recursion - level 0

⇒ A function calls itself to solve smaller versions of the same problem

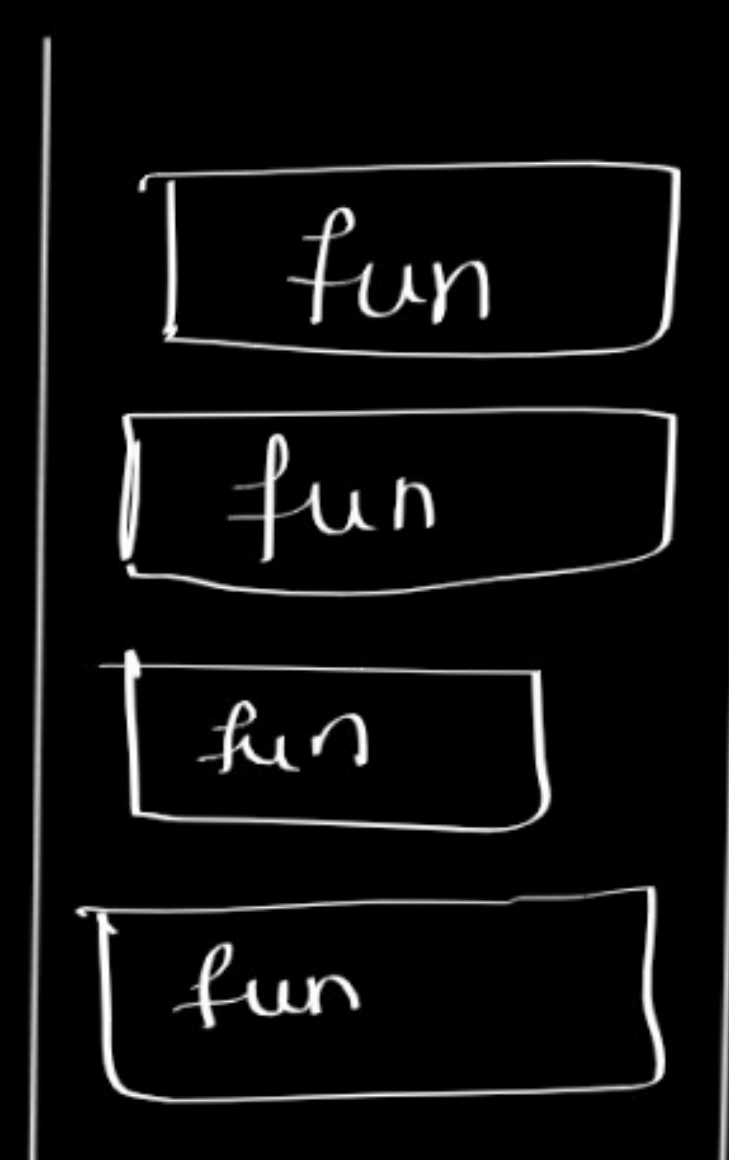
① Base case: To stop recursion when condition met.

② Recursive case: calls the same function but with simplified (or) reduced version.

```
function fun(num)
{
  console.log(num);
  num = num - 1;
  fun(num);
}
```

→ This function will go into an loop and get a **Stackoverflow error**

```
let num = 5;
fun(num);
```



Call stack

Whenever function gets called it will be pushed into call stack

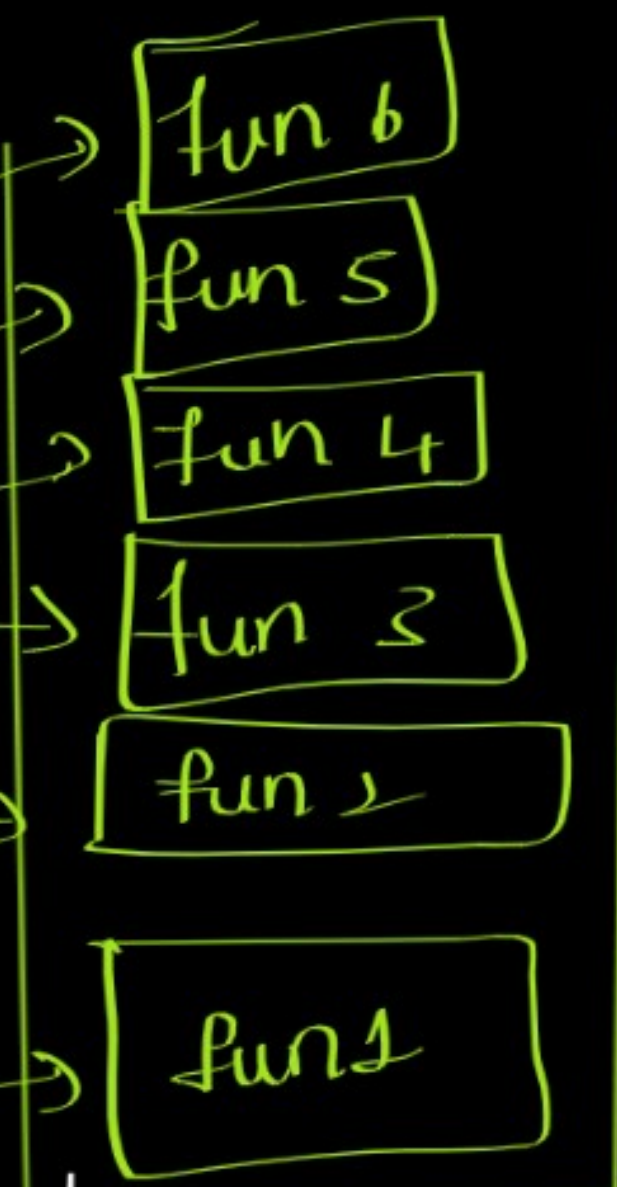
But when there is no base case it will call many times when the capacity of call stack exceeds it will give stack overflow error.

```
function fun(num)
```

```
{
  if (num < 1) return;
  console.log(num);
  fun(--num);
}
```

```
let num = 5;
fun(num);
```

0 < 1 (true) → stops executing next lines of code



console

5
4
3
2
1

Printing from n to 1

when base case meets each and every function context will pop off.

```
let num = 5;
function fun(x) {
  if (x > num) return;
  console.log(x);
  fun(++x);
}
fun(1);
```

console

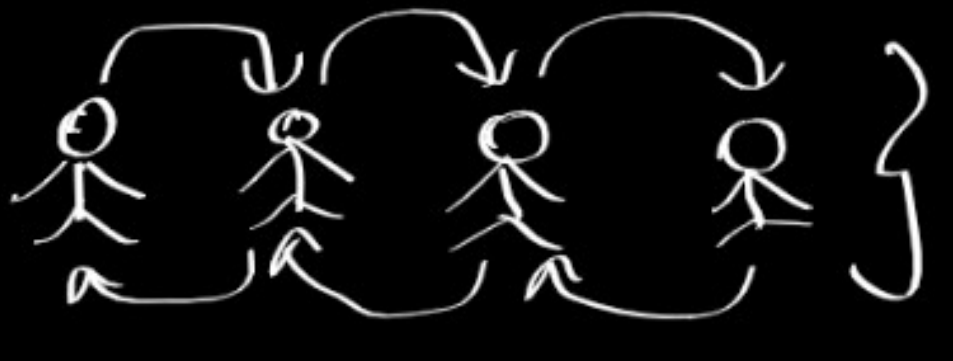
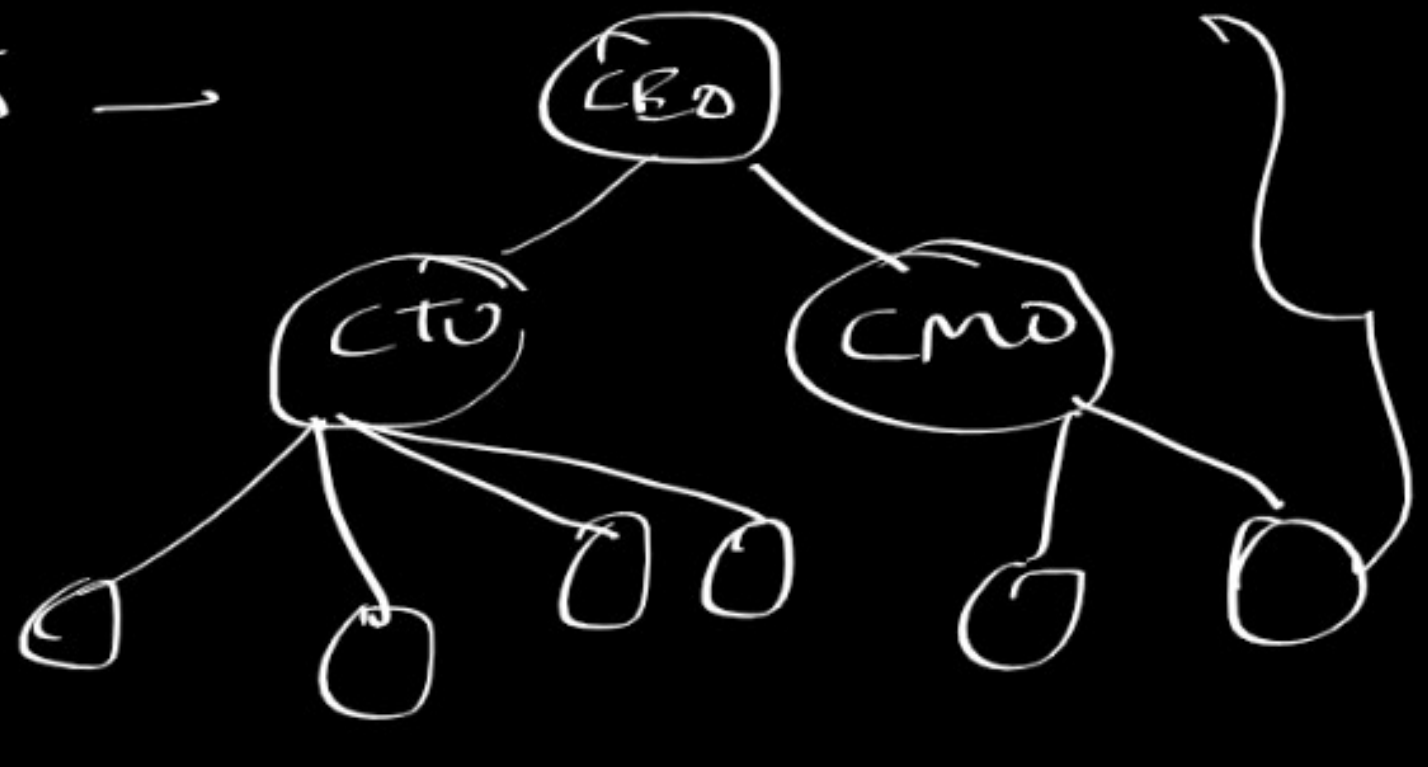
1
2
3
4
5

x	num	cond ⁿ
1	5	1 > 5 (false)
2		2 > 5 (false)
3		3 > 5 (false)
4		4 > 5 (false)
5		5 > 5 (false)
6		6 > 5 (true)

Base condition met, recursion stops here

Printing from 1 to n

Real-life examples:

- ① Queue of people  finding no. of people in queue
- ② Comment threads — reddit comment threads
- ③ Organisational hierarchies —  finding total no. of employees

Sum of first N Numbers

$$N=5 \rightarrow 1+2+3+4+5 \text{ (or) } 5+4+3+2+1$$

Code:

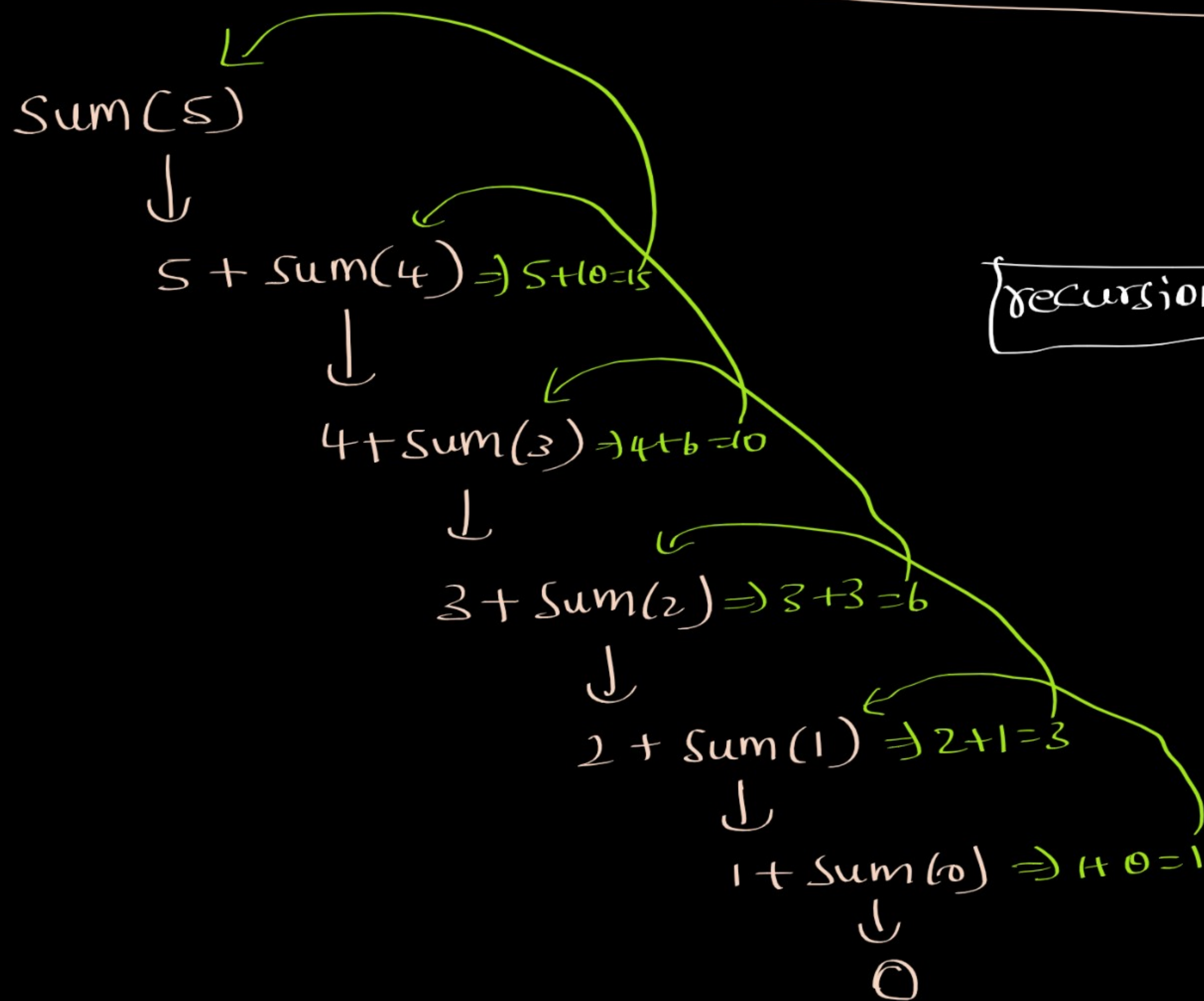
function sum(num)

```
{ if (num === 0) return 0;  
  return num + sum(num-1);  
}
```

sum(5)

Call Stack:

sum(0)	→ return 0
sum(1)	→ return 1 + sum(0) ⇒ 1+0=1
sum(2)	→ return 2 + sum(1) ⇒ 2+1=3
sum(3)	→ return 3 + sum(2) ⇒ 3+3=6
sum(4)	→ return 4 + sum(3) ⇒ 4+6=10
sum(5)	→ return 5 + sum(4) ⇒ 5+10=15



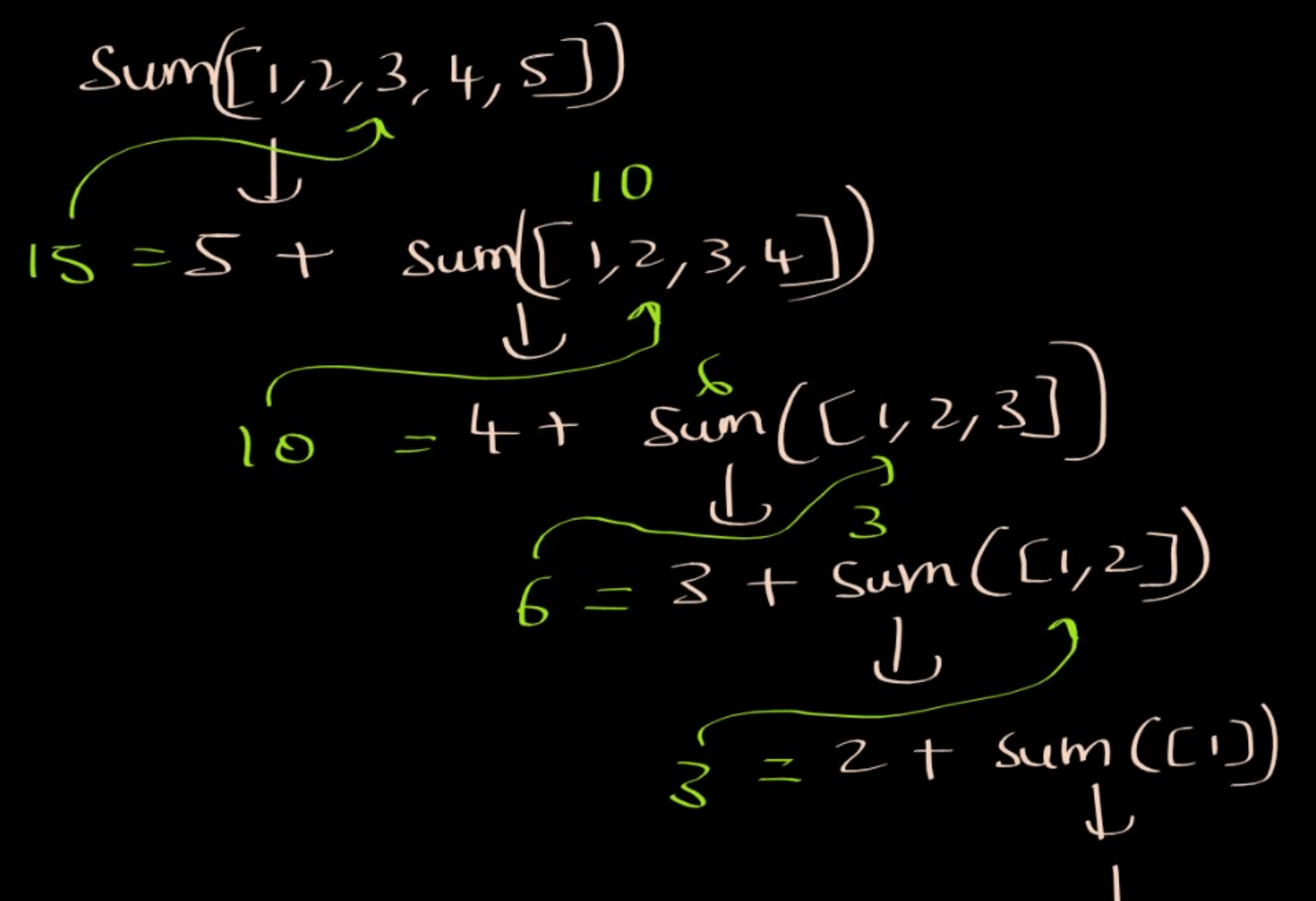
recursion tree

Sum of All numbers in an Array

$$[1, 2, 3, 4, 5] \rightarrow \text{o/p: 15}$$

function sum(index)

```
{ if (index === 0) return arr[0] (or) arr[index];  
  return arr[index] + sum(index-1);  
}
```



Sum of all odd numbers in an array

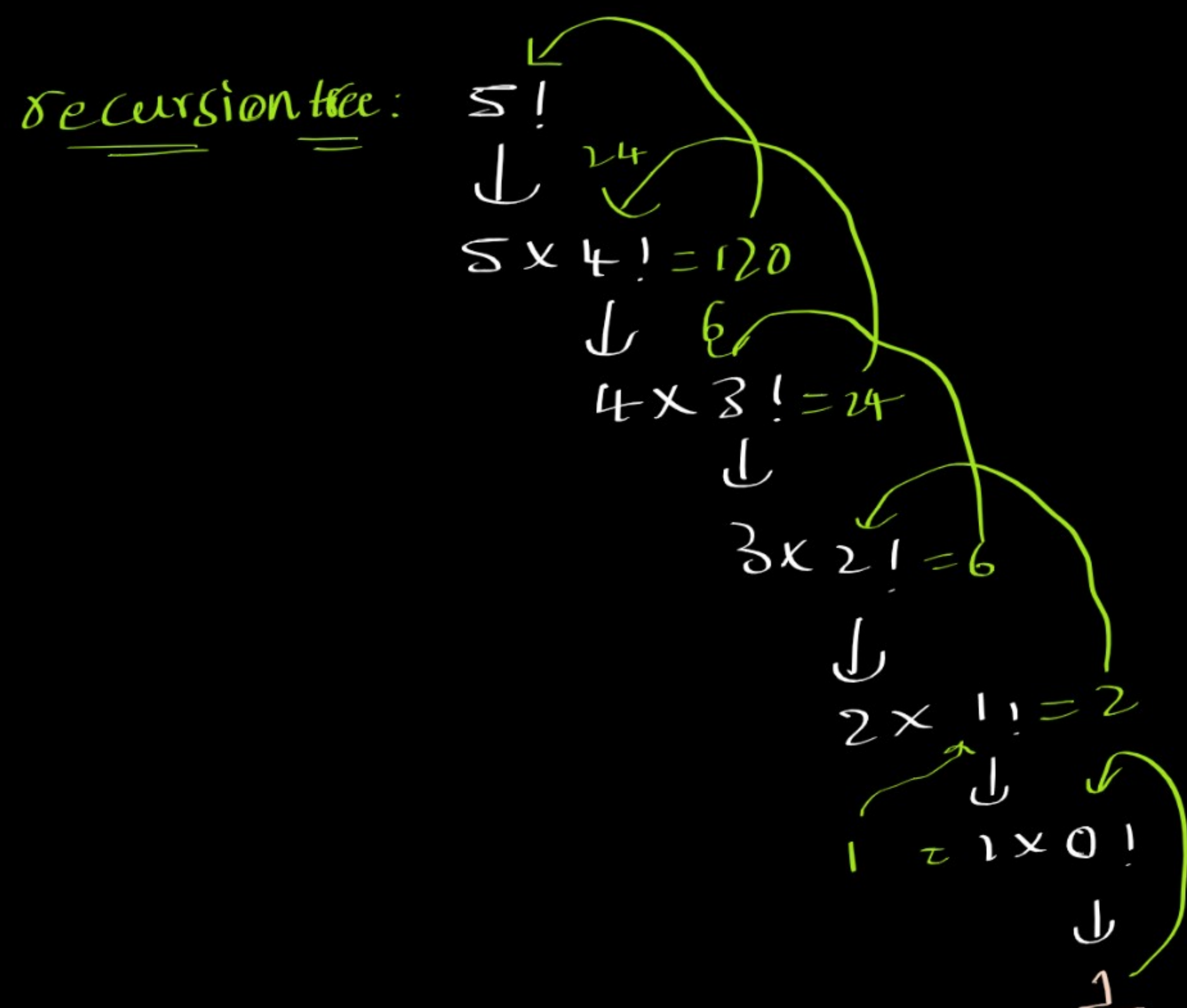
```
function sumOfArray(index) {
  if (index === 0) return arr[index] % 2 === 1 ? arr[0] : 0;
  return arr[index] % 2 === 1
    ? arr[index] + sumOfArray(--index)
    : 0 + sumOfArray(--index);
}
```

arr = [2, 1, 3, 4, 7, 4]

index = 5	arr[index] % 2 === 1	
index = 5	false	Sum(4)
index = 4	true	1 = 7 + Sum(3)
index = 3	false	Sum(2)
index = 2	true	4 = 3 + Sum(1)
index = 1	true	1 = 1 + Sum(0)
index = 0	false	0

factorial of n

$n * \text{factorial}(n-1)$



Code:

```
function factorial(num)
{
  if (num === 0) return 1;
  return num * factorial(num-1);
}
```

Power of two

- 16 → 16/2 → 8/2 → 4/2 → 2/2 → 1 → if num is power of two then it will eventually become 1 when continuously divided.
- num < 1 (or) num is odd → Not power of two.

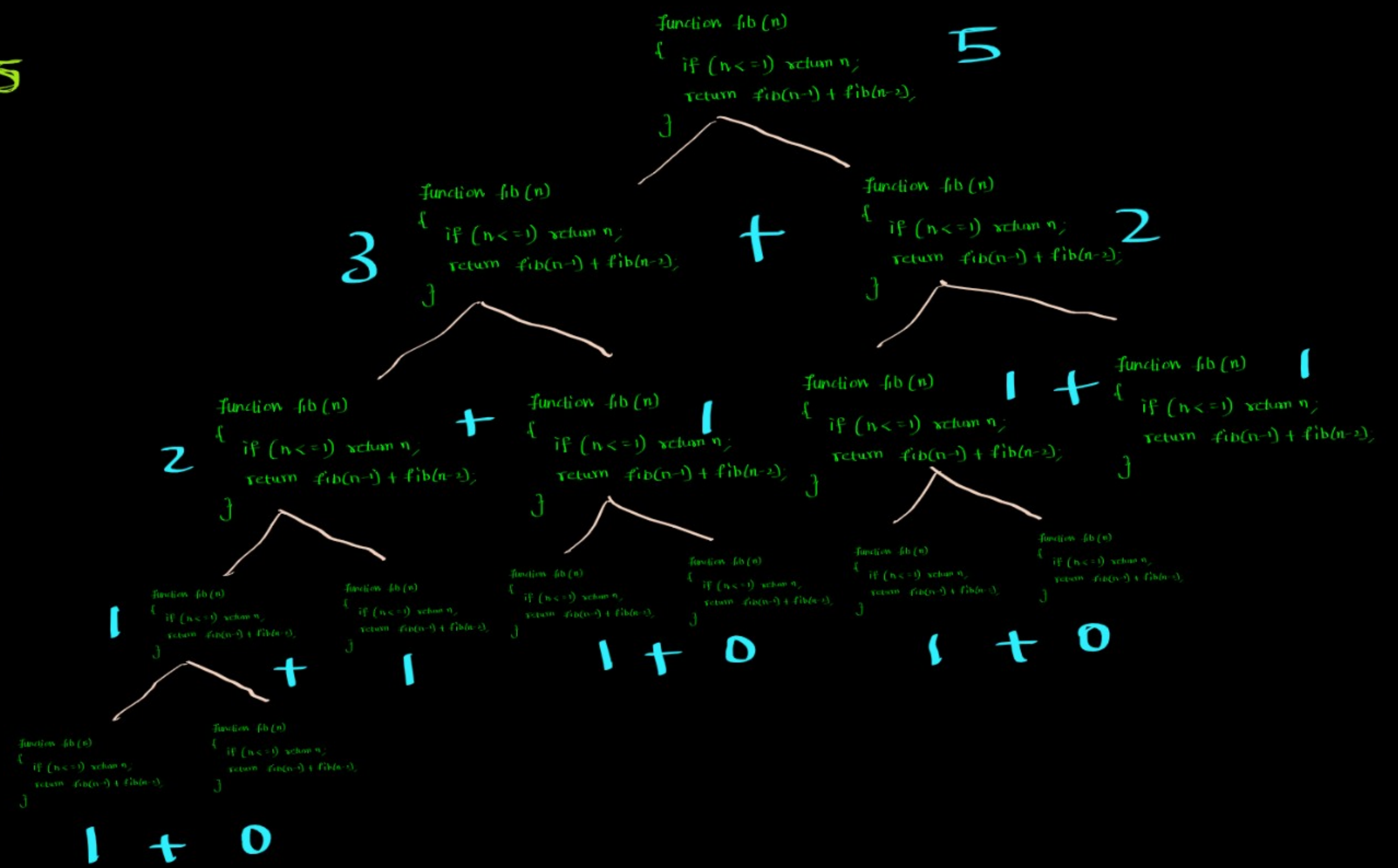
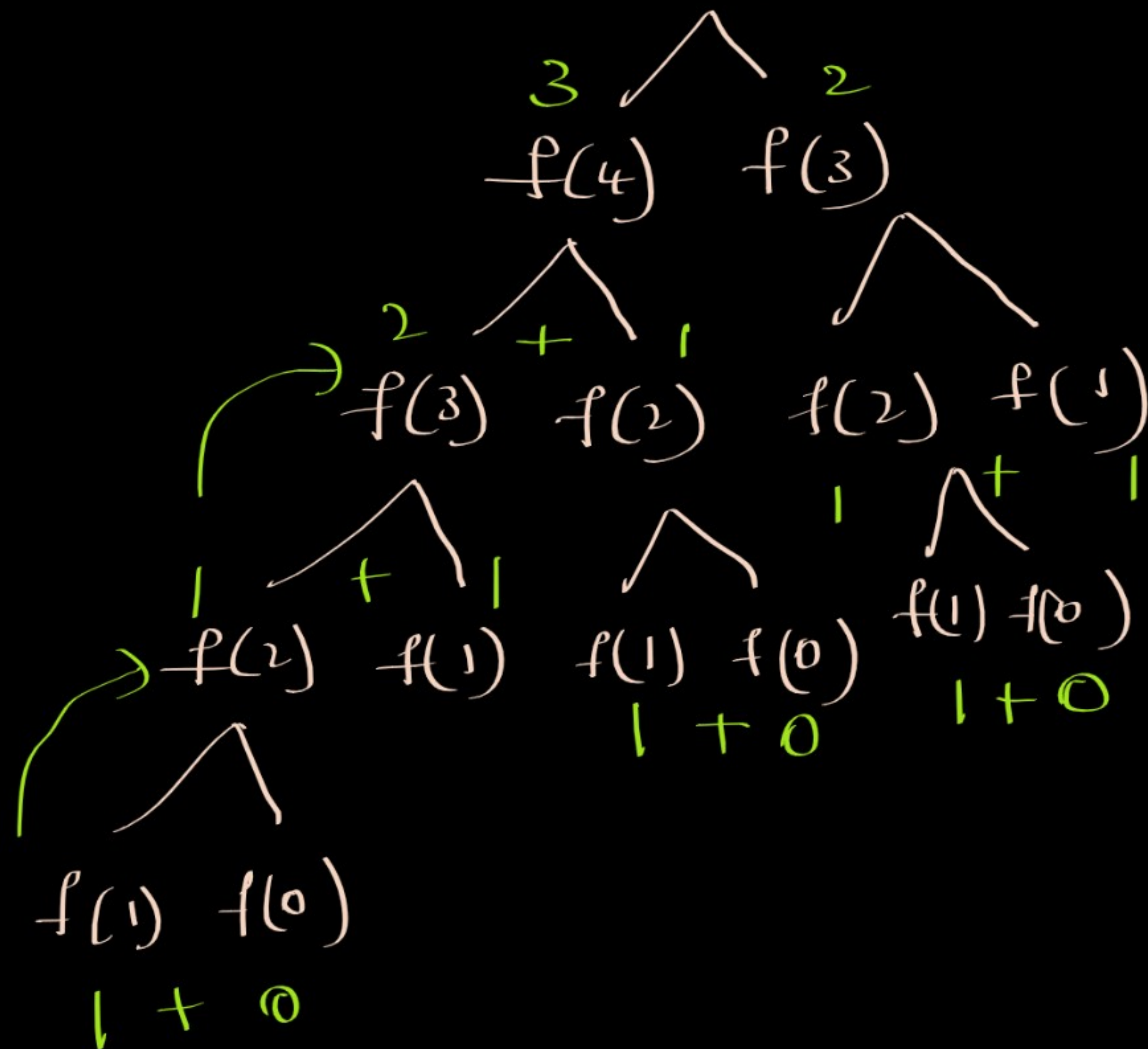
```
function powerOfTwo(num)
{
  if (num === 1) return true;
  else if (num < 1 || num % 2 !== 1) return false;
  return powerOfTwo(num/2);
}
```


Recursion Masterclass

Fibonacci number: $f(0)=0$; $f(1)=1$ $f(n) = f(n-1) + f(n-2)$

0	1	2	3	4	5
0	1	—	—	—	—

Recursion tree: $f(5) \rightarrow 3+2=5$



$n \rightarrow 2n \rightarrow 4n \rightarrow 8n \rightarrow 16n \dots 1$

$n(2 \times 2 \times 2 \dots) = 1$

$n * 2^n = 1 \Rightarrow \boxed{O(2^n)}$